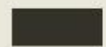


Listas Encadeadas

Estrutura de dados 2



Discente: Vinicius Rodrigues

Professor: Adriano Antunes Prates



A Ordem Oculta dos Dados

Vetores, Listas Encadeadas e a engenharia invisível que move a computação.

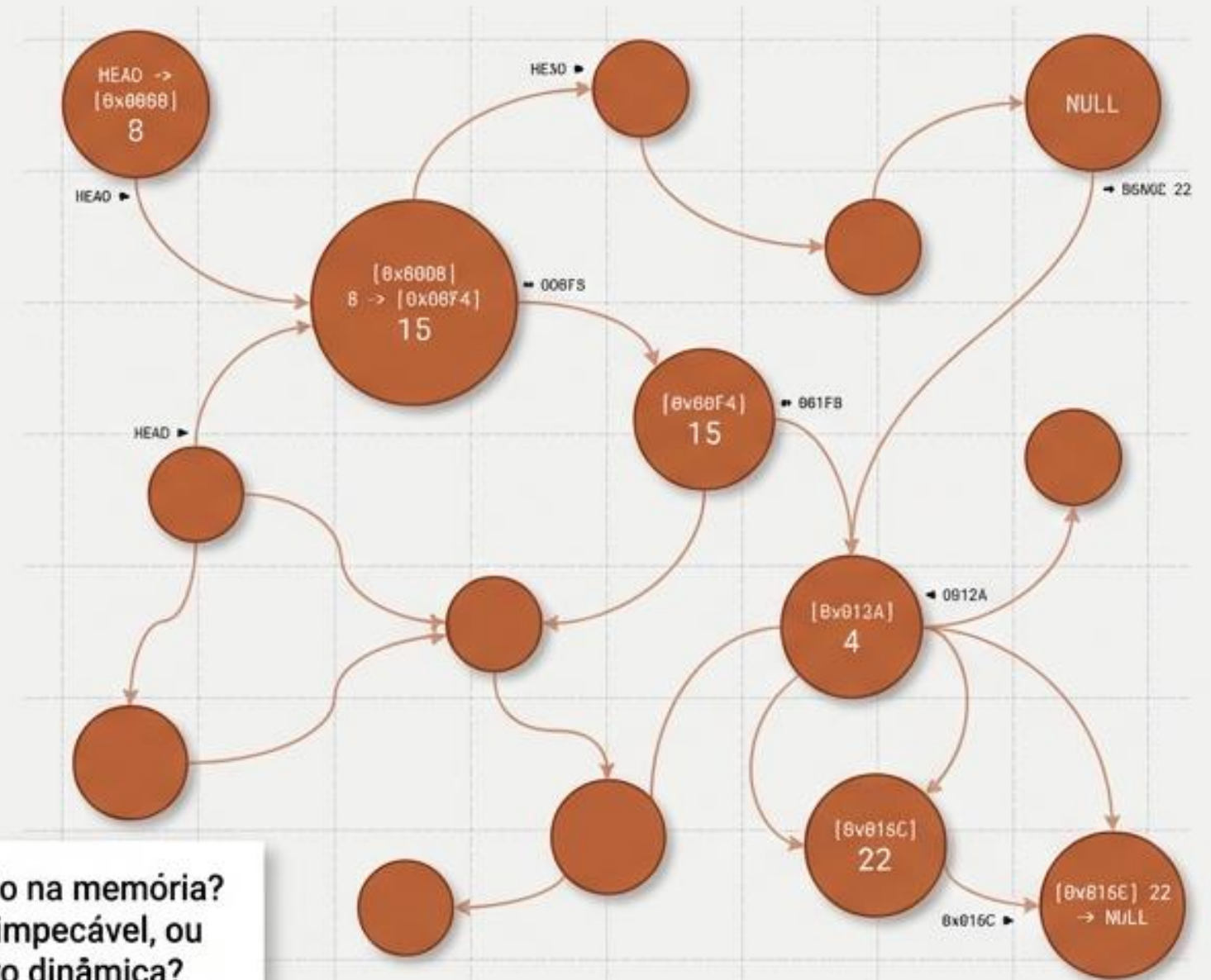
VETORES (ARRAYS): ESTRUTURAS SEQUENCIAIS E RÍGIDAS

	[8v0040] 12	[8x0044] 35	[8x8048] 7	[8x804C] 21	[8v864C] 14	[8x8146] 15	[8v812A] 28	[8v816C] 21
0x8040	[0x0040] 12	[8x0044] 35	[8x0048] 7	[8v804C] 21				
0x8044								
0x8048								
8x804S								
8x864C								
8x864C								
8x8B4C								

Como org
Como un

**Como organizar a informação na memória?
Como uma fila de exército impecável, ou
como uma caça ao tesouro dinâmica?**

LISTAS ENCADEADAS: CAMINHOS ORGÂNICOS E FLEXÍVEIS



A Fundação: A Lista Linear Sequencial



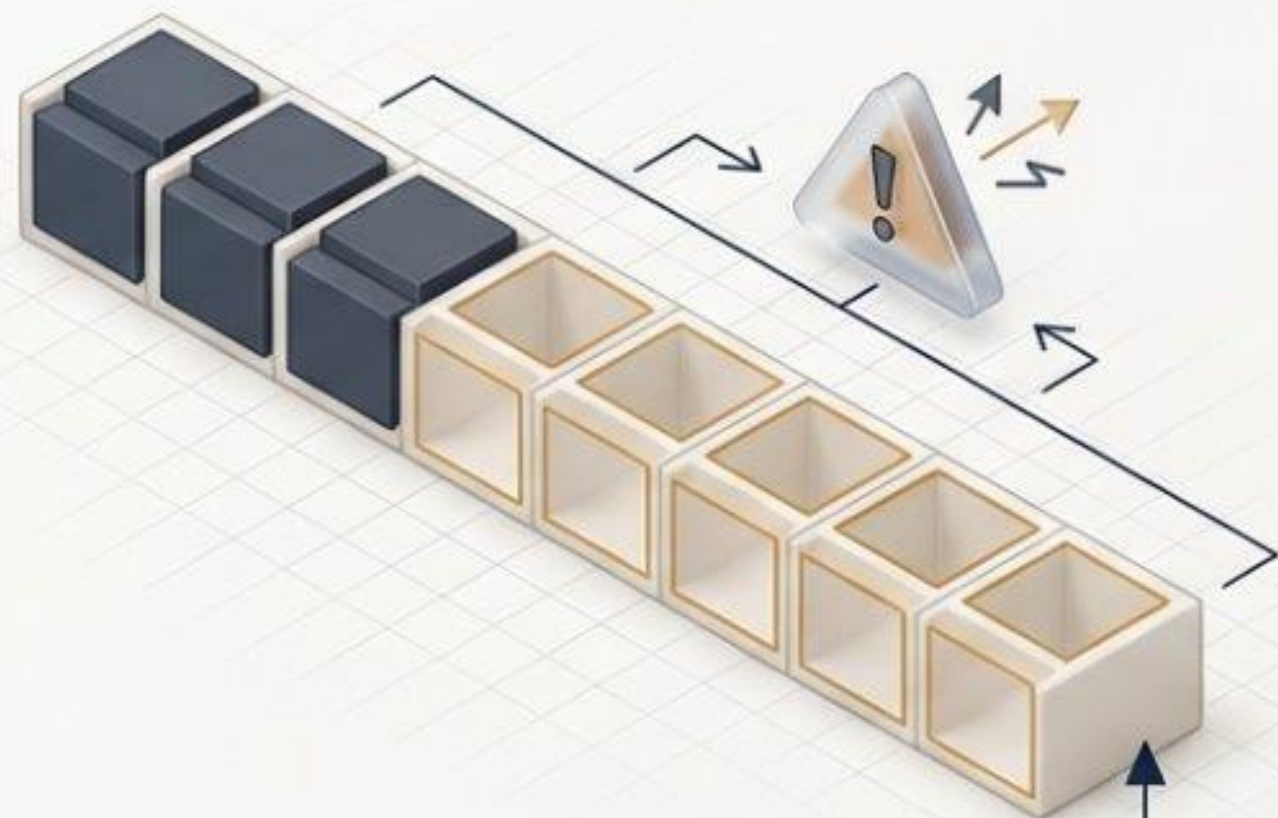
CONCEITO CHAVE

Ordem Perfeita. Cada elemento possui um predecessor e um sucessor exatos.

EXPLICAÇÃO

A mágica do vetor (array) é a simplicidade. A forma como organizamos a lista na nossa cabeça é idêntica à forma como ela é gravada fisicamente nos chips de memória. Vizinhos na lógica são vizinhos na física.

O custo oculto do tamanho fixo



Memória alocada à toa

Adivinhar o tamanho máximo antecipadamente gera um vasto desperdício de recursos.



Falta de espaço (Overflow)

Subestimar o tamanho necessário resulta em quebra imediata do programa.

0 Preço da Perfeição: 0 Calcanhar de Aquiles

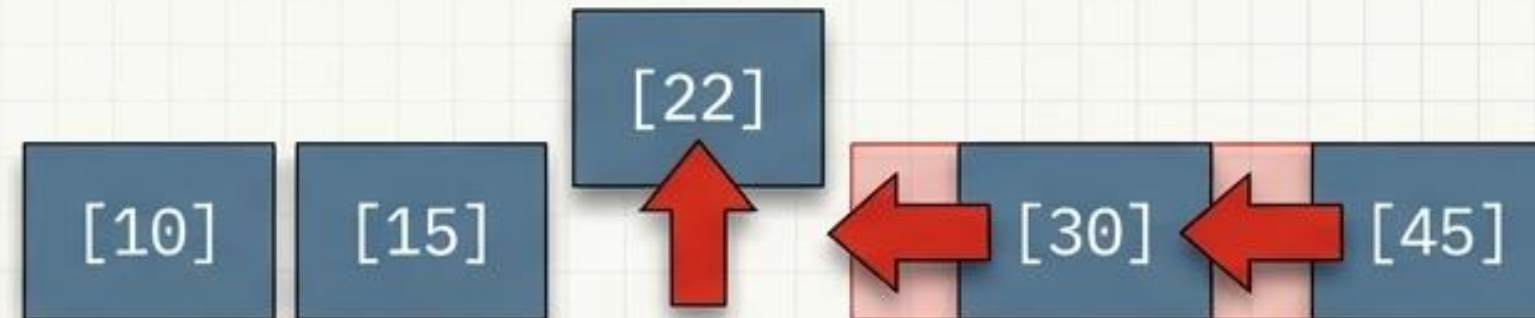
Passo 1:
Faltam espaços



Passo 2:
0 Empurra-Empurra



Passo 3: Puxando
para tapar o buraco

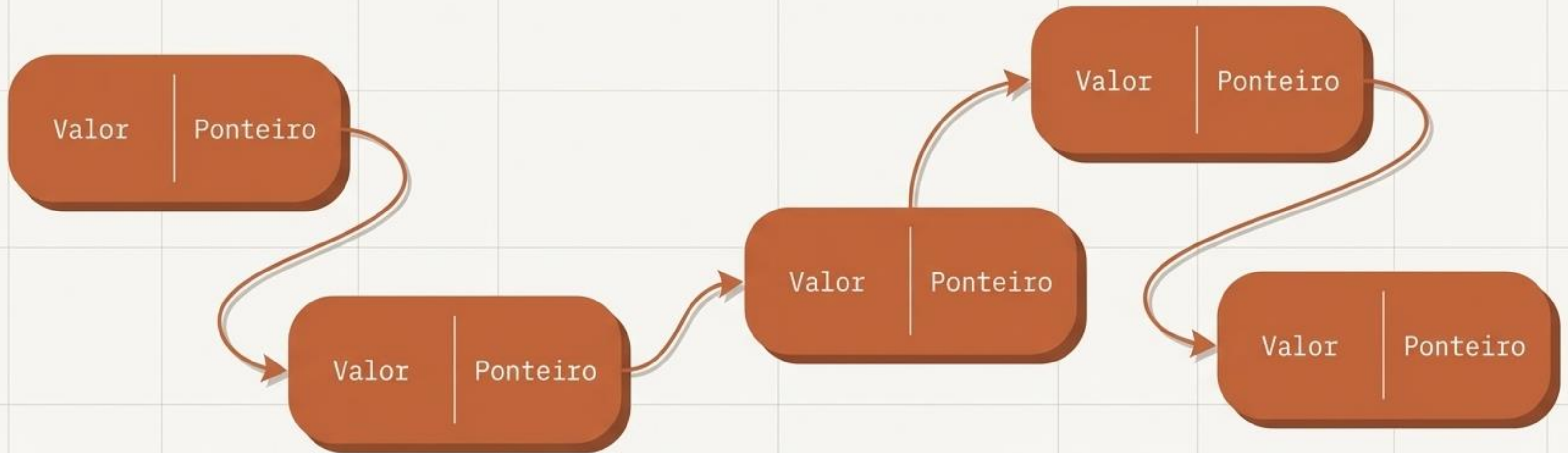


Custo da Inflexibilidade.

Modificar o meio de uma lista sequencial exige um trabalho gigantesco. O computador perde um tempo precioso deslocando memória física em vez de processar dados. Tamanho fixo e rigidez estrutural criam gargalos de performance.

A Solução Elegante: Listas Encadeadas

Ordem Lógica $\neq$ Ordem Física



Na lista encadeada, a memória está intencionalmente espalhada. Os elementos habitam qualquer espaço livre do sistema.

! O Ponteiro. A ordem não é mais garantida pela proximidade física, mas por uma pista. Cada elemento carrega o endereço exato do próximo item, criando uma caça ao tesouro inquebrável.

A Cirurgia Limpa

Inserir



Remover

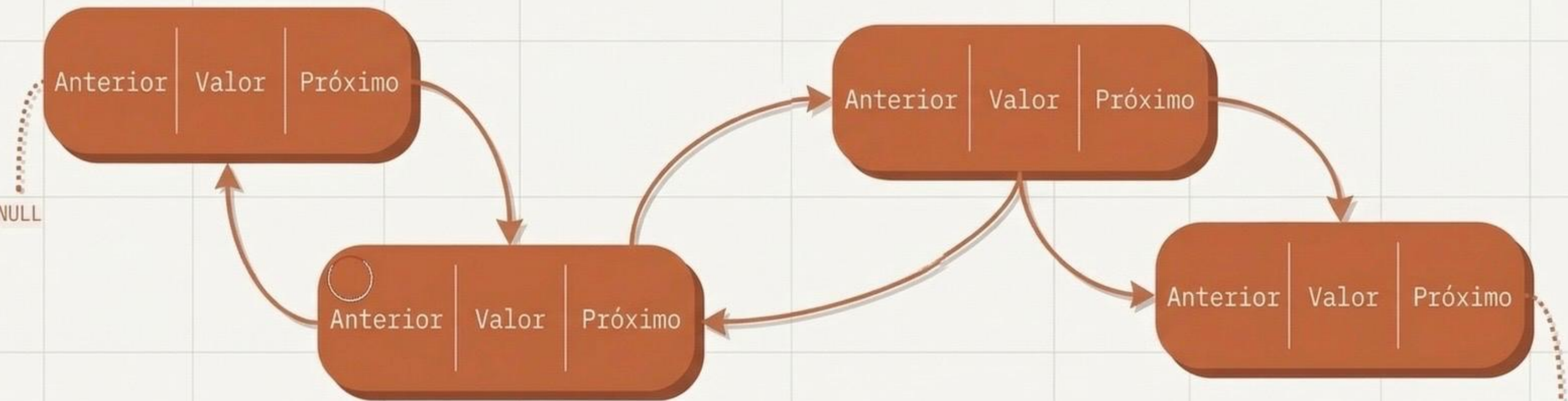


Inserir ou remover dados agora é apenas uma questão de redirecionar dois fios (ponteiros).

Ninguém mais se mexe. Independentemente de haver 10 ou **10 milhões** de itens na lista, o esforço do computador é mínimo e idêntico. O gargalo do empurra-empurra foi eliminado.

A Evolução do Conector: Listas Duplamente Encadeadas

Navegação Bidirecional



Na lista duplamente encadeada, o nó é aprimorado. Cada elemento habita qualquer espaço livre, mas seus nós agora têm dois guias: um para o **Próximo** item e um para o **Anterior** item.



Os Dois Ponteiros. A ordem física permanece aleatória. No entanto, o nó agora tem dois guias. Isso permite que você rastreie não apenas para onde está indo (Próximo), mas de onde veio (Anterior), abrindo novas possibilidades de controle e navegação.

0 Duelo de Arquiteturas

	Lista Sequencial	Lista Encadeada
Acesso / Busca	 Imediato (Direto)	 Lento (Passo a passo)
Modificação	 Custo altíssimo (Deslocamento)	 Custo zero (Troca de ponteiros)
Gestão de Memória	 Engessada (Tamanho fixo)	 Flexível (Cresce por demanda)

0 ganho absoluto em flexibilidade custa o superpoder do acesso instantâneo. A busca binária morre na Lista Encadeada.

A Escolha Estratégica: Não Existe Bala de Prata



Dicionário Online

Carregamento estático + Buscas frenéticas.
A escolha inquestionável: **Lista Sequencial**
(Poder da Busca Binária).



Função Desfazer (Ctrl+Z)

Inserção/Remoção constante da última ação.
A escolha ideal: **Lista Encadeada** (Velocidade
de Modificação e tamanho infinito).

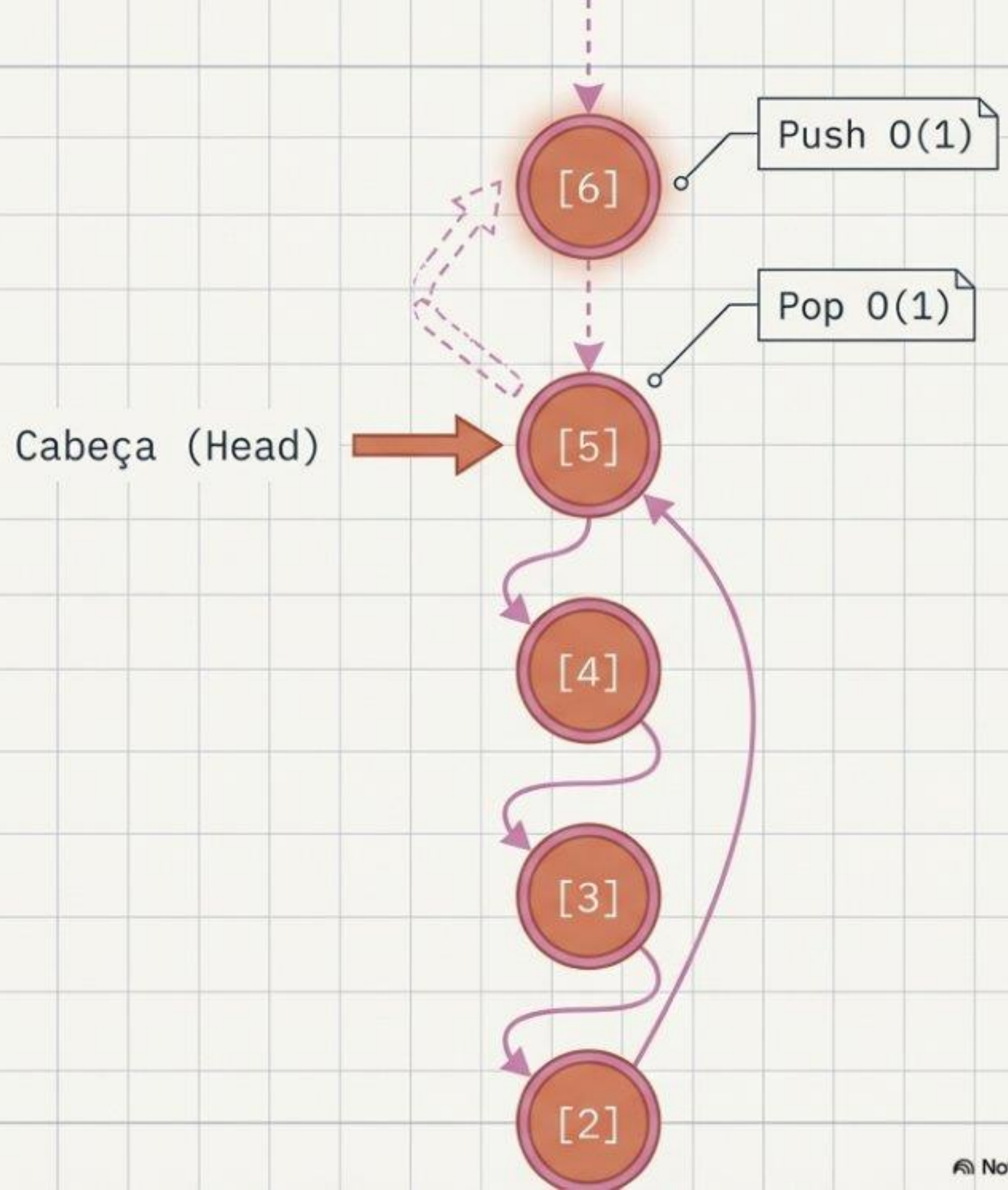
A verdadeira pergunta não é qual estrutura é superior. É descobrir se o seu problema exige a velocidade absoluta do acesso direto ou a flexibilidade de uma cirurgia sem dor. Fazer essa escolha corretamente é o que separa um código bom de uma arquitetura genial.

Reinventando Pilhas (LIFO)

Usando listas encadeadas para Pilhas, o topo é a Cabeça.

Adicionar (Push) ou remover (Pop) afeta apenas o primeiro ponteiro.

Resultado: tempo constante e crescimento infinito sem limite pré-definido.

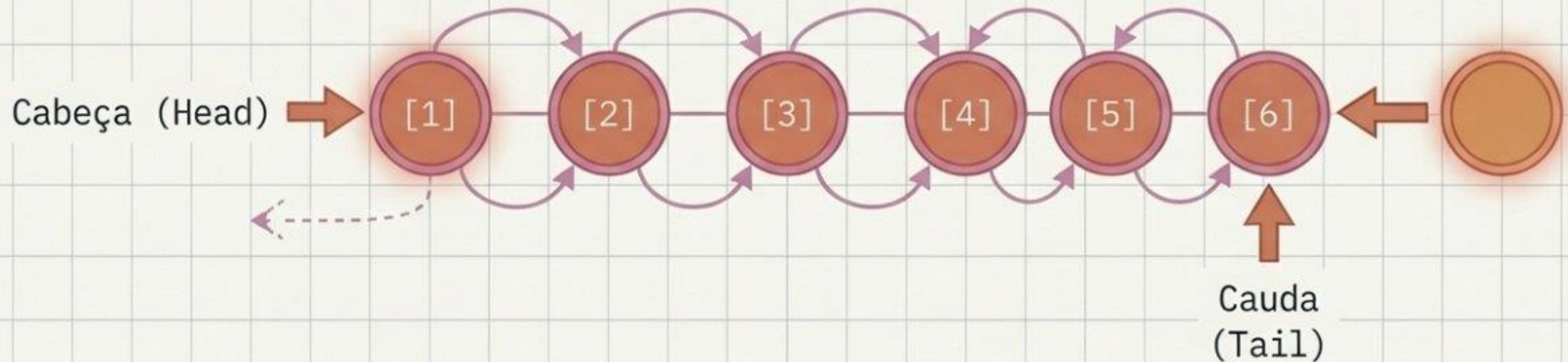




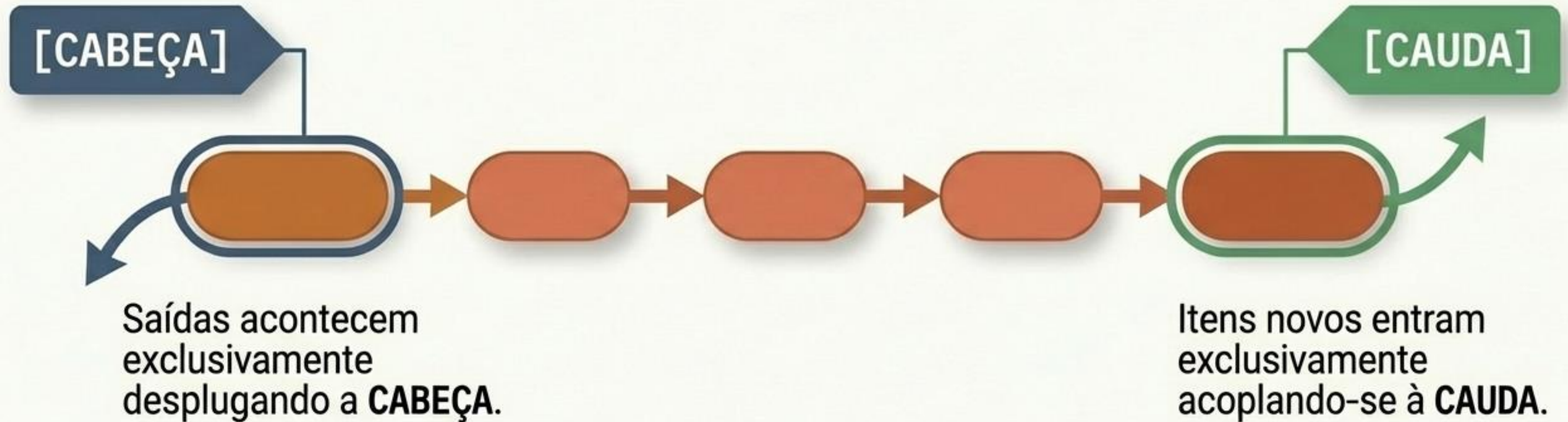
A Arquitetura das Filas (FIFO)

Para filas ultra-eficientes, implementamos dois ponteiros. Elementos entram pela Cauda e saem pela Cabeça.

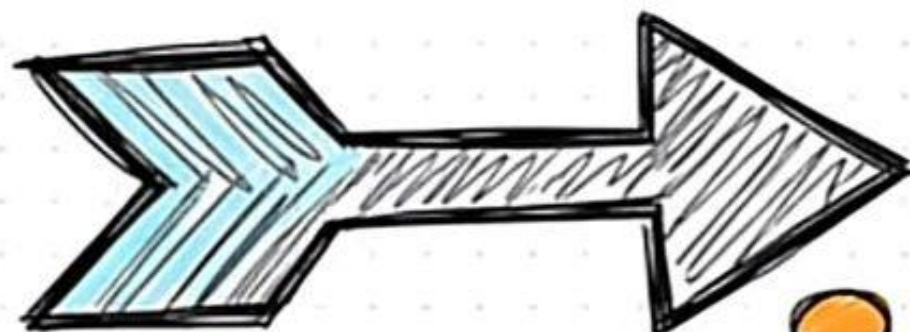
Ambas as operações ocorrem em tempo constante, sem deslocar o resto da fila.



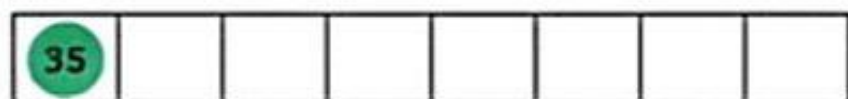
Reinventando Estruturas: A Fila (Queue) com Dois Motores



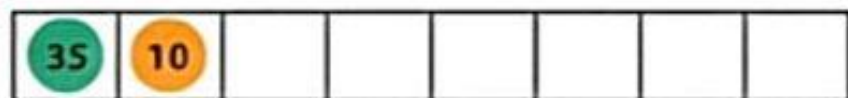
Ao contrário da Pilha, a Fila exige ação nos dois extremos. A solução? Um segundo ponteiro mestre. Com um rastreador em cada ponta, a fila flui perfeitamente sem necessitar de um único deslocamento interno de memória.



Enqueue: 35



Enqueue: 10



Enqueue: 94, 12, 45



Dequeue



Dequeue

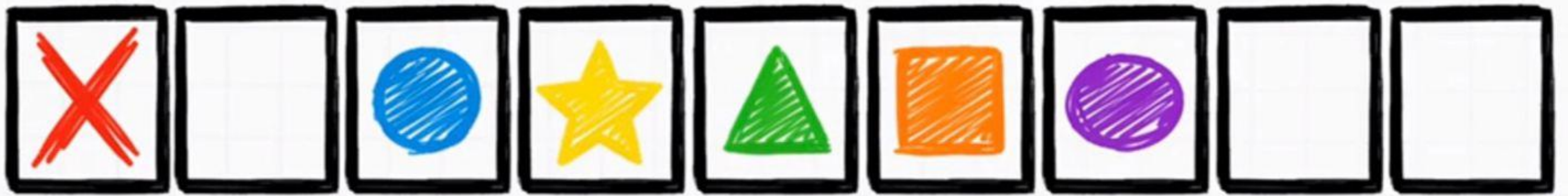


Enqueue: 32

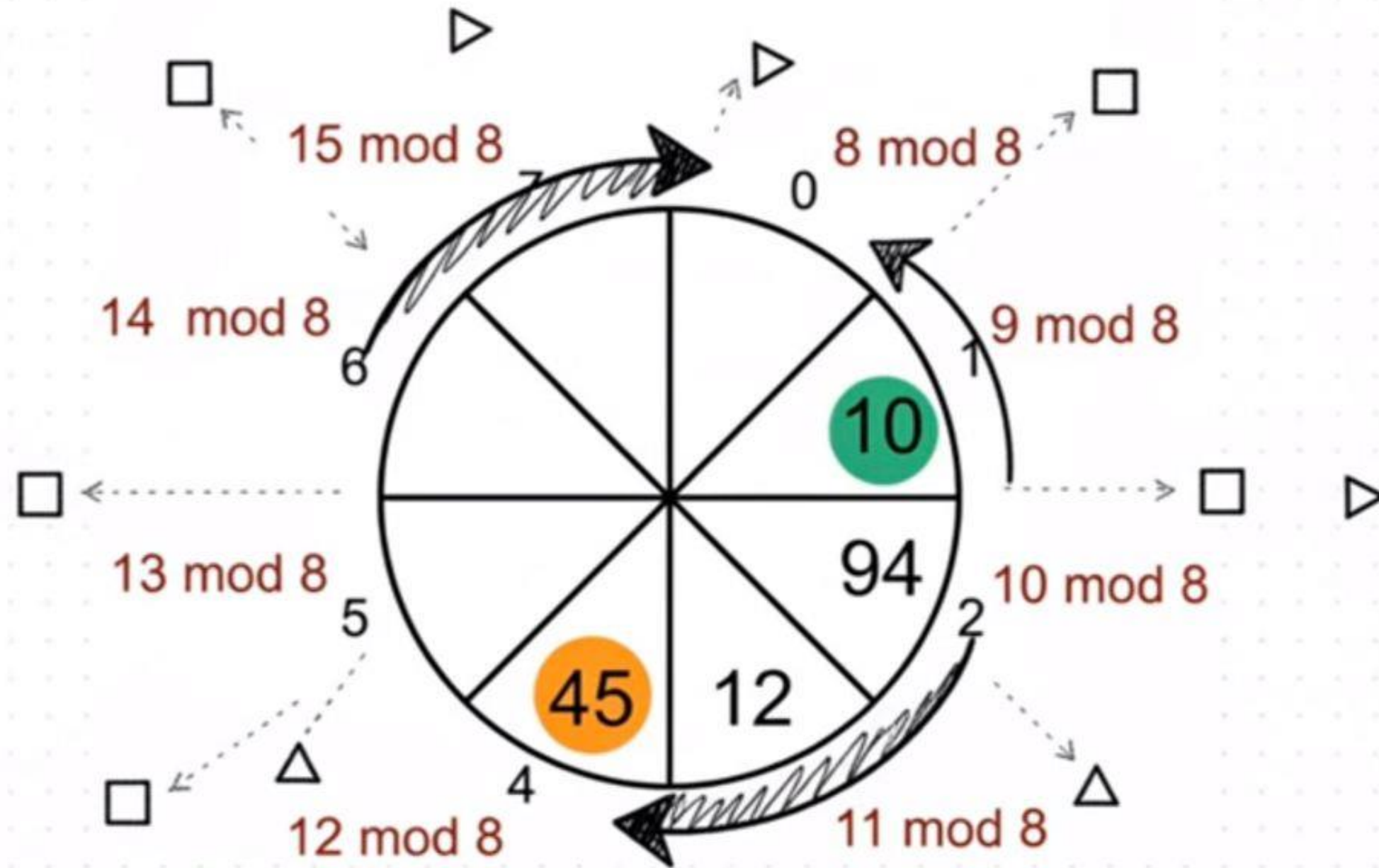


À medida que itens são adicionados e removidos, a fila **caminha** pelo array.





Número de Elementos: 8



E se tratássemos o array como um círculo, conectando o final ao começo?